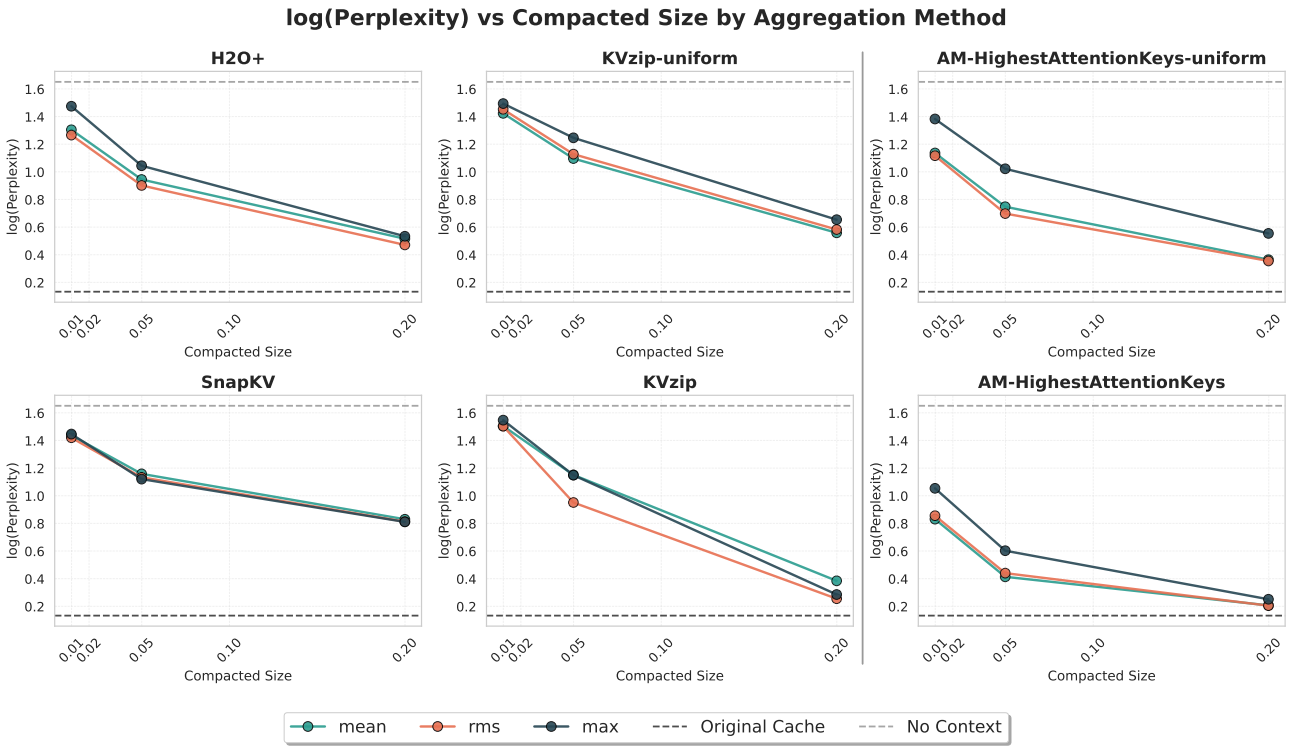

Figure 9. **Effect of aggregation function on key selection.** We plot accuracy versus compaction ratio for four baseline methods and our Highest Attention Keys method on a subset of 20 QuALITY articles using Qwen3-4B. All methods select the top-t keys based on aggregated attention scores over a shared set of reference queries.

Two notable deviations from this trend are **Cartridges** and **summarization**. Cartridges tends to achieve slightly *lower* reconstruction loss than methods with similar downstream accuracy, consistent with its objective: it performs end-to-end latent-cache optimization using a distillation-style loss that targets the original model's behavior, directly optimizing token-level fidelity. In contrast, summarization often yields *higher* reconstruction loss than methods with similar accuracy: it discards substantial token-level detail but can preserve (or even emphasize) the subset of information most useful for answering comprehension questions. In other words, summarization can be a good task heuristic despite being a poor reconstruction mechanism.

Overall, these results support two conclusions: (i) reconstruction loss is a useful proxy for comparing compaction methods *within* families that aim to preserve attention behavior, and (ii) it should not be treated as a universal predictor of downstream performance when methods fundamentally change the objective. We further note that self-log-perplexity in long-context evaluations can be skewed on many language models due to entropy blowup as generations grow longer (Braverman et al., 2020; Cao et al., 2025). The perplexity numbers reported here are for Qwen3-4B on QuALITY (contexts < 10k tokens), where we did not observe such entropy blowup, though we did observe this at longer context lengths.

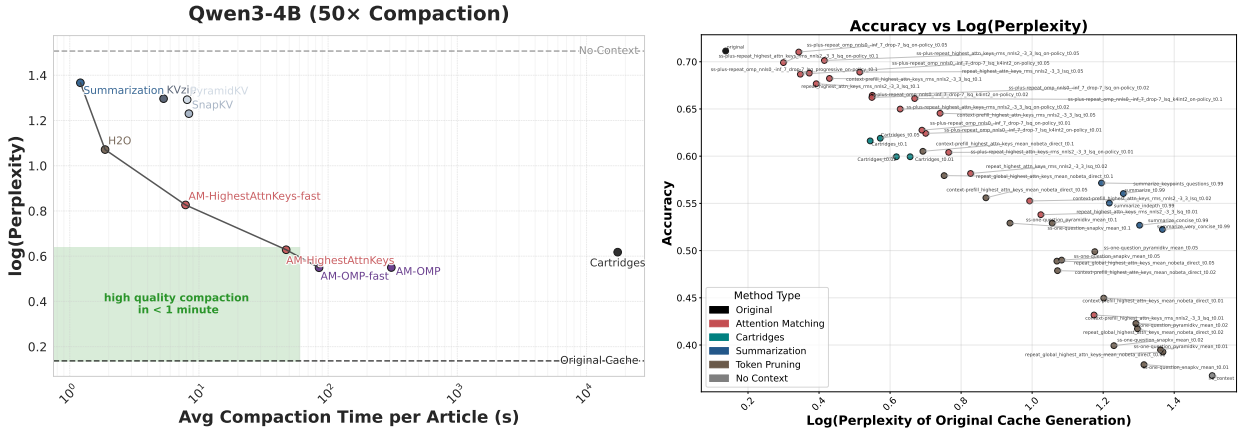

Figure 10. **Reconstruction loss vs. downstream accuracy. Left:** Loss-based analogue of Figure 1, instead reporting log-perplexity on original-cache generations. **Right:** Downstream multiple-choice accuracy plotted against this loss across various methods and compaction ratios.

F.3. Extension: Online Compaction

For long-horizon reasoning, we may want to compact *mid-trajectory* so the model can continue decoding after producing thousands of intermediate tokens.

Phys Len	**Eff Len**	**AIME** (/30)	**Notes**
2048	2048	1.25	
4096	4096	7.75	
8192	8192	13	
With online compaction			
2048	4096	8	$\leq$2 compactions
2048	8192	13	$\leq$6 compactions

Table 7. AIME performance (pass@1, averaged over 4 runs) with and without mid-trajectory compaction. *Effective sequence length* is the total number of decoded tokens, while *physical sequence length* is the maximum physical size of the KV cache. Each compaction reduces the size of the entire KV cache (including previously compacted portions) by 50%, so $\leq$2 and $\leq$6 compactions correspond to approximately $2\times$ and $4\times$ more decoded tokens, respectively. For implementation simplicity, we do not enable non-uniform compaction or on-policy queries. We use AM-HighestAttnKeys-Fast. We believe more tuned methods (e.g. with higher compaction ratios, compacting only the most recently generated tokens, etc.) may improve results.

We conduct a proof-of-concept evaluation of online compaction on **AIME 2025** with Qwen3-4B. For each problem, we cap the *physical* sequence length during reasoning to a fixed budget (Phys Len). Whenever the model reaches this budget, we compact the entire context *except* for the most recent 20 tokens, and then continue decoding until reaching a target *effective* decoding length (Eff Len). We compare against standard decoding to the same effective length without any compaction. To control token budgets, when the model reaches the maximum context length we inject `\nI need to respond with the answer.\n</think>Final Answer:>` (following Muennighoff et al. (2025)) and decode the final answer.

Overall, these results suggest that online compaction can effectively decouple reasoning depth from physical context limits. Despite shrinking the KV cache mid-trajectory up to six consecutive times, the model achieves performance comparable to standard decoding at the same effective length, indicating that essential reasoning state is largely preserved. Our study uses a simple uniform compaction scheme and always compacts the entire context, rather than, for example, freezing previously compacted portions or selectively compacting recent tokens. We believe that more careful tuning could enable much greater scaling. We present these results primarily to demonstrate the strong empirical stability of reasoning under repeated mid-trajectory compaction.

Finally, extending online compaction to multi-turn settings, such as compacting long tool call outputs, remains an interesting application for future work.